

5. Наследяване и полиморфизъм в C++. Видове наследяване. Предефиниране на методи (Overriding) и оператори (Overloading). Примерен код на C++.

1. Въведение

Обектно-ориентираното програмиране (ООП) стъпва върху четири основни принципа: **абстракция, капсулация, наследяване и полиморфизъм**.

Първите два описват как се изгражда и предпазва вътрешното състояние на отделния клас, докато наследяването и полиморфизмът описват отношенията между класовете и позволяват повторно използване и разширяване на вече написан код.

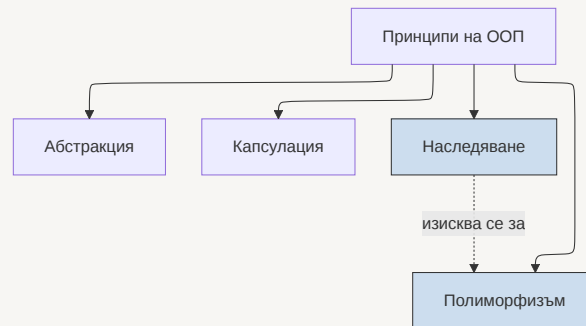
Наследяването дава механизъм, при който нов клас (производен) поема структурата и поведението на съществуващ клас (базов) и ги допълва или променя. Полиморфизмът ("много форми") позволява един и същ интерфейс да отговаря с различно поведение в зависимост от конкретния тип на обекта по време на изпълнение. Двата принципа са тясно свързани: в C++ истинският динамичен полиморфизъм е невъзможен без йерархия от наследени класове и виртуални функции.

```
graph TD
    OOP[Принципи на ООП]
    OOP --> A[Абстракция]
```

```

ООР --> Е[Капсулация]
ООР --> I[Наследяване]
ООР --> P[Полиморфизъм]
I -. изисква се за .-> P
style I fill:#cde,stroke:#333
style P fill:#cde,stroke:#333

```



2. Наследяване

2.1 Същност

Наследяването е механизъм, чрез който един клас (наричан наследник или подклас) се изгражда на основата на друг клас (базов или родителски). Производният клас автоматично получава всички член-данни и член-функции на базовия (с някои ограничения за достъпа) и може:

- да добавя нови член-данни и член-функции
- да предефинира (override) наследени методи
- да използва наследената функционалност без да я преписва

Отношението, което наследяването моделира, се нарича "е вид" (is-a).

Например Куче е вид Животно, затова `Dog` наследява `Animal`. Това го отличава от отношението "има" (has-a), което се моделира чрез композиция (член-обект).

2.2 Синтаксис

```
class Base {};  
  
class Derived : public Base {};
```

Форматът е:

```
class <производен_клас> : <спецификатор_за_достъп> <базов_клас> { ... };
```

2.3 Прост пример

```
class Animal {  
protected:  
    string name;  
  
public:  
    Animal(const string& n) : name(n) {}  
    void breathe() const {  
        cout << name << " диша" << endl;  
    }  
};  
  
class Dog : public Animal {  
public:  
    Dog(const string& n) : Animal(n) {} // извикване на конструктора на базовия клас  
    void bark() const {  
        cout << name << " лае" << endl; // достъп до protected член  
    }  
};  
  
int main() {  
    Dog d("Щапо");
```

```

d.breathe(); // наследен метод от Animal
d.bark(); // метод на Dog

return 0;
}

```

3. Видове наследяване

Наследяването в C++ се разглежда по 2 начина:

- според спецификатора за достъп (как се променя видимостта на наследените членове)
- според структурата на йерархията (колко и как са свързани класовете).

3.1 Според спецификатора за достъп

Спецификаторът за достъп пред името на базовия клас определя как нивата на достъп `public` / `protected` / `private` на базовите членове се пренасят в производния клас.

Достъп в базовия клас	<code>public</code> наследяване	<code>protected</code> наследяване	<code>private</code> наследяване
<code>public</code> член	остава <code>public</code>	става <code>protected</code>	става <code>private</code>
<code>protected</code> член	остава <code>protected</code>	остава <code>protected</code>	става <code>private</code>
<code>private</code> член	недостъпен	недостъпен	недостъпен

- `private` членовете на базовия клас никога не са пряко достъпни от производния - независимо от вида наследяване. Достъп до тях е възможен само през публичните/защитените методи на базовия клас
- `public` наследяване е най-честото и единствено то изразява чисто отношение "е вид". Външен код вижда производния обект и през интерфейса на базовия

- `protected` и `private` наследяване изразяват по-скоро “реализирано чрез” (implemented-in-terms-of) и се използват рядко; те скриват базовия интерфейс от външния свят

3.2 Според структурата на йерархията

Единично наследяване

Един производен клас има точно един базов клас. Това е най-простата и често срещана форма.

```
classDiagram
    class A
    class B
    A <|-- B
```



Многостепенно наследяване

Клас наследява друг клас, който на свой ред е наследник на трети - образува се верига.

```
classDiagram
    class Animal
    class Mammal
    class Dog
```

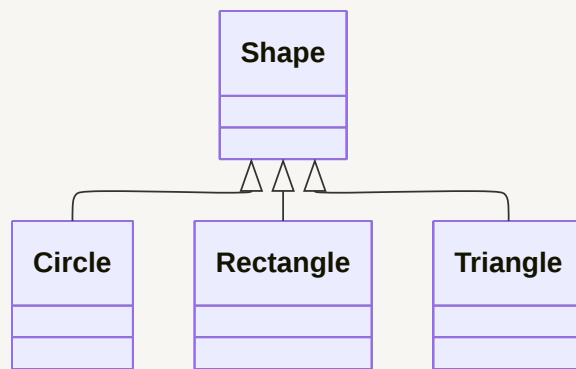
```
Animal <|-- Mammal  
Mammal <|-- Dog
```



Йерархично наследяване

Няколко производни класа споделят един общ базов клас.

```
classDiagram
    class Shape
    class Circle
    class Rectangle
    class Triangle
    Shape <|-- Circle
    Shape <|-- Rectangle
    Shape <|-- Triangle
```



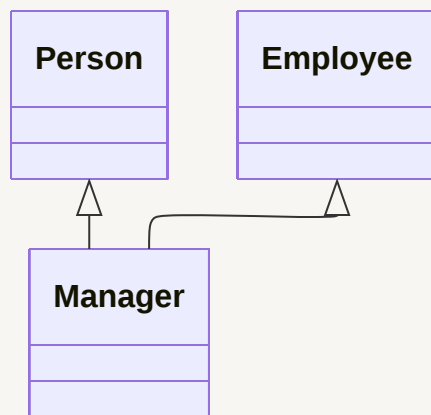
Множествено наследяване

Един клас наследява едновременно два или повече класа. C++ е сред малкото езици, които го поддържат пряко.

```
class Person { };
class Employee { };

class Manager : public Person, public Employee {
    // наследява и от двата класа
};
```

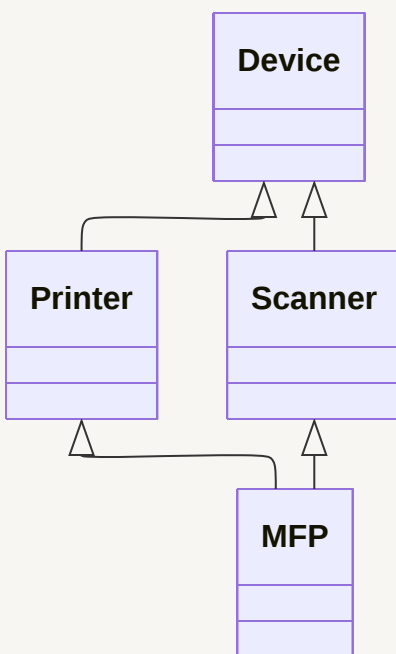
```
classDiagram
    class Person
    class Employee
    class Manager
    Person <|-- Manager
    Employee <|-- Manager
```



Хибридно наследяване и “проблемът с диаманта”

Когато множественото и многостепенното наследяване се комбинират, може да възникне ситуация, в която един базов клас се наследява по два различни пътя. Това е т.нар. **diamond problem** (проблем на диаманта): производният клас получава две копия на членовете на най-горния базов клас, което води до двусмисленост.

```
classDiagram
class Device
class Printer
class Scanner
class MFP
Device <|-- Printer
Device <|-- Scanner
Printer <|-- MFP
Scanner <|-- MFP
```



Решението в C++ е **виртуалното наследяване** (**virtual**), което гарантира, че споделяният базов клас има само едно общо копие:

```
class Device {  
public:  
    int id;  
};  
  
class Printer : virtual public Device { };  
class Scanner : virtual public Device { };  
  
// Сега MFP има само едно копие на Device::id  
class MFP : public Printer, public Scanner { };
```

4. Конструктори и деструктори при наследяване

При създаване на обект от производния клас конструкторите се изпълняват "отгоре надолу" - първо този на базовия клас, после на производния. Деструкторите се изпълняват в обратен ред - първо на производния, после на базовия.

```
class Base {
public:
    Base() { cout << "Конструктор Base\n"; }
    ~Base() { cout << "Деструктор Base\n"; }
};

class Derived : public Base {
public:
    Derived() { cout << "Конструктор Derived\n"; }
    ~Derived() { cout << "Деструктор Derived\n"; }
};

int main() {
    Derived d;
    return 0;
}

/* Изход:
Конструктор Base
Конструктор Derived
Деструктор Derived
Деструктор Base
*/
```

Ако базовият клас няма конструктор по подразбиране, производният трябва изрично да извика подходящ конструктор в инициализацията списък:

```
Derived() : Base(args) { ... }.
```

Важно правило: Ако клас е предназначен да бъде базов и работи през указатели/референции, неговият деструктор трябва да е виртуален (`virtual ~Base();`). В противен случай при `delete` на указател от тип базов клас, сочещ

към произведен обект, **деструкторът на производния клас** няма да се извика и ще възникне изтичане на ресурси.

5. Полиморфизъм

Полиморфизъм означава "много форми" - възможността един и същ интерфейс (име на функция, оператор, указател) да предизвиква различно поведение според конкретния тип. В C++ той бива два основни вида:

- Статичен (компиляционен) полиморфизъм - кой код ще се изпълни се решава по време на компилация. Реализира се чрез предефиниране на функции (function overloading), предефиниране на оператори и шаблони.
- Динамичен (изпълнителен) полиморфизъм - решението се взема по време на изпълнение в зависимост от реалния тип на обекта. Реализира се чрез **виртуални функции** и работа през указатели или референции към базовия клас.

6. Предефиниране на методи (Overriding)

6.1 Същност

Overriding (предефиниране) означава производният клас да предостави нова реализация на метод, който вече съществува в базовия клас, със същата сигнатура (същото име, същите параметри, същия тип на резултата). Това е основата на динамичния полиморфизъм.

6.2 Виртуални функции и механизмът на действие

За да работи късното свързване, методът в базовия клас трябва да е обявен с ключовата дума `virtual`. Тогава при извикване през указател/референция към базовия клас C++ проверява **реалния тип** на обекта и извиква съответната версия.

```
class Shape {  
public:  
    virtual double area() const { return 0.0; }    // виртуал
```

```

ен метод
    virtual ~Shape() {} // виртуал
ен деструктор
};

class Circle : public Shape {
    double r;
public:
    Circle(double radius) : r(radius) {}
    double area() const override { // override – предефиниран
        return 3.14159 * r * r;
    }
};

class Square : public Shape {
    double a;
public:
    Square(double side) : a(side) {}
    double area() const override { // override – предефиниран
        return a * a;
    }
};

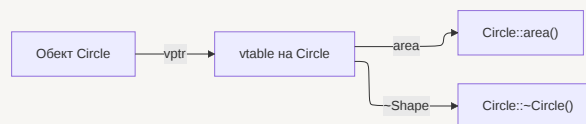
int main() {
    Shape* shapes[] = { new Circle(2.0), new Square(3.0) };
    for (Shape* s : shapes) {
        cout << "Лице: " << s->area() << endl; // правилната
        версия според реалния тип
        delete s;
    }
    return 0;
}
/* Изход:
Лице: 12.5664

```

Технически C++ реализира това чрез таблица на **виртуалните функции** (**vtable**): всеки клас с виртуални методи има скрита таблица с указатели към реалните им реализации, а всеки обект пази скрит указател (vptr) към таблицата на своя клас. При виртуално извикване програмата минава през $vptr \rightarrow vtable \rightarrow \text{реалния метод}$.

graph LR

```
obj[Обект Circle] -->|vptr| vt[vtable на Circle]
vt -->|area| ca["Circle::area()"]
vt -->|~Shape| cd["Circle::~~Circle()"]
```



6.3 Ключови уми **override** и **final**

- **override** изрично заявява, че методът предефинира виртуален метод от базовия клас. Ако сигнатурата случайно не съвпада, компилаторът дава грешка.
- **final** забранява по-нататъшно предефиниране на метода (или наследяване на класа)

```
class Base { public: virtual void f(); };

class Derived : public Base {
public:
    void f() override final;    // предефинира и забранява по-
    нататъшно предефиниране
};
```

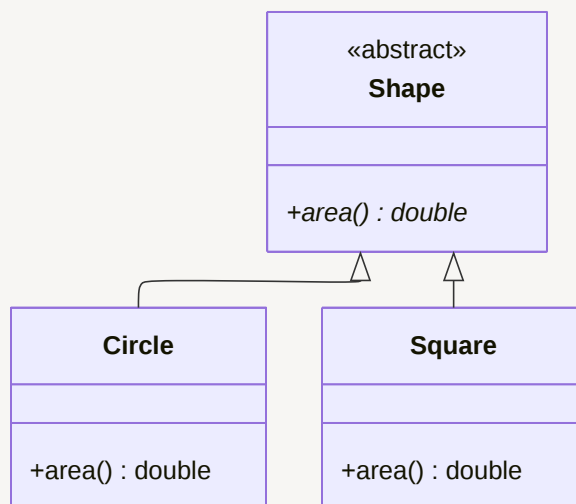
6.4 Абстрактните класове и чисто виртуални функции

Когато базовият клас няма смислена реализация на метод, той се обявява като чисто виртуален с `= 0`. Клас с поне една чисто виртуална функция е **абстрактен** - не може да се създават обекти от такъв клас, служи само за интерфейс.

```
class Shape {
public:
    virtual double area() const = 0;    // чисто виртуална функция
    virtual ~Shape() {}
};

// Не може да се създаде обект като Shape s;
// Всеки наследник е длъжен да реализира area(), иначе остава абстрактен
```

```
classDiagram
    class Shape {
        <<abstract>>
        +area() double*
    }
    class Circle {
        +area() double
    }
    class Square {
        +area() double
    }
    Shape <|-- Circle
    Shape <|-- Square
```



7. Предефиниране на оператори (Overloading)

7.1 Същност

Предефинирането на оператори позволява на стандартните оператори (+, -, *, ==, <<, [], () и др.) да придобият смисъл за потребителски типове. Това е форма на статичен полиморфизъм: операторът се придобива с нова функция, която компилаторът избира според типовете на операндите.

Операторът се дефинира чрез специална функция с име `operator<символ>`. Тя може да е:

- член-функция на класа - левият операнд е обектът `this`
- глобална/приятелска (friend) функция - когато левият операнд не е от нашия клас (например при `<<` и `ostream`)

7.2 Правила и ограничения

- Не може да се създават нови символи на оператори - предефинират се съществуващи
- Не могат да се предефинират `::`, `.`, `.*`, `?:` и `sizeof`

- Запазват се приоритетът, асоциативността и броят операнди на оператора
- Поне един от операндите трябва да е от потребителски тип

7.3 Примерен код (клас за комплексни числа)

```
class Complex {
    double re, im;
public:
    Complex(double r = 0, double i = 0) : re(r), im(i) {}

    Complex operator+(const Complex& other) const {
        return Complex(re + other.re, im + other.im);
    }

    bool operator==(const Complex& other) const {
        return re == other.re && im==other.im;
    }

    friend ostream& operator<<(ostream& out, const Complex&
c) {
        out << c.re << " " << c.im << "i";
        return out;
    }
};

int main() {
    Complex a(2, 3), b(1, -4);
    Complex c = a + b;           // използва operator+
    cout << "a = " << a << endl; // използва operator<<
    cout << "b = " << b << endl;
    cout << "a + b = " << c << endl;
    cout << a == b << endl; // използва оператор==
    return 0;
}
```

8. Обединяващ примерен код

```
// Абстрактен базов клас
class Employee {
protected:
    string name;
    double baseSalary;

public:
    Employee(const string& n, double s) : name(n), baseSalary(s) {}
    virtual double salary() const = 0;           // чисто виртуален – overriding
    virtual ~Employee() {}

    // предефиниран оператор << (overloading)
    friend ostream& operator<<(ostream& out, const Employee& e) {
        out << e.name << ": " << e.salary() << " лв.";
        return out;
    }
};

// Единично наследяване
class Developer : public Employee {
    double bonus;
public:
    Developer(const string& n, double s, double b)
        : Employee(n, s), bonus(b) {}
    double salary() const override {           // предефиниране
        return baseSalary + bonus;
    }
};
```

```

// Многостепенно наследяване: Manager е вид Developer
class Manager : public Developer {
    int teamSize;
public:
    Manager(const string& n, double s, double b, int t)
        : Developer(n, s, b), teamSize(t) {}
    double salary() const override {                // предефиниран
ане
        return Developer::salary() + teamSize * 100;
    }
};

int main() {
    vector<Employee*> staff = {
        new Developer("Иван", 2000, 500),
        new Manager("Мария", 2500, 600, 5)
    };

    for (const Employee* e : staff)
        cout << *e << endl;                        // динамичен
полиморфизъм + operator<<

    for (Employee* e : staff) delete e;                // виртуален
деструктор се грижи за чистенето
    return 0;
}

/* Изход:
    Иван: 2500 лв.
    Мария: 3600 лв.
*/

```

```

classDiagram
    class Employee {
        <<abstract>>

```

```
#string name
#double baseSalary
+salary() double*
+operator<<()
}
class Developer {
    -double bonus
    +salary() double
}
class Manager {
    -int teamSize
    +salary() double
}
Employee <|-- Developer
Developer <|-- Manager
```

